

```

CREATE TRIGGER trg_EmployeeCountUpdate
ON employee
AFTER INSERT, DELETE
AS
BEGIN
    -- Update the EmployeeCount table with current total number of
    employees
    UPDATE EmployeeCount
    SET emp_count = (SELECT COUNT(*) FROM employee);
END;

```

ALTERNATIVE APPROACH (MORE EFFICIENT FOR LARGE TABLES):

If you want to avoid counting the entire table each time, you can increment/decrement the count:

sql

```

CREATE TRIGGER trg_EmployeeCountUpdate
ON employee
AFTER INSERT, DELETE
AS
BEGIN
    DECLARE @rowCount INT;

    -- Get the number of affected rows
    SET @rowCount = (SELECT COUNT(*) FROM inserted) + (SELECT COUNT(*)
    FROM deleted);

    -- If rows were inserted, increment the count
    IF EXISTS (SELECT 1 FROM inserted)
    BEGIN
        UPDATE EmployeeCount
        SET emp_count = emp_count + (SELECT COUNT(*) FROM inserted);
    END

    -- If rows were deleted, decrement the count
    IF EXISTS (SELECT 1 FROM deleted)
    BEGIN
        UPDATE EmployeeCount
        SET emp_count = emp_count - (SELECT COUNT(*) FROM deleted);
    END
END;

```

HOW TO TEST IT:

Step 1: Initial Setup

sql

```
-- Assuming EmployeeCount already has a row  
INSERT INTO EmployeeCount (emp_count) VALUES (10);
```

Step 2: Insert a new employee

sql

```
INSERT INTO employee (emp_id, name, dept, salary)  
VALUES (11, 'Jane Smith', 'HR', 65000);  
  
-- Check the EmployeeCount table  
SELECT * FROM EmployeeCount;
```

Result after INSERT:

emp_count
11

Step 3: Delete an employee

sql

```
DELETE FROM employee WHERE emp_id = 1;  
  
-- Check the EmployeeCount table  
SELECT * FROM EmployeeCount;
```

Result after DELETE:

emp_count
10

IMPORTANT NOTES:

1. Initial row requirement: The `EmployeeCount` table must have at least one row initially for the UPDATE to work. You can initialize it with:
2. Handling multiple rows: The trigger handles batch operations (multiple INSERTs or DELETEs in one statement)
3. Alternative with MERGE: If you also have UPDATE operations that change department (not relevant here), you might need additional logic
4. Performance: The first approach using `SELECT COUNT(*)` is simpler and maintains accuracy even if the trigger fails or if manual changes are made to the employee table
5. Error handling: You may want to add error handling to ensure the EmployeeCount table always has exactly one row:

sql

-- Ensure only one row exists

```
IF NOT EXISTS (SELECT 1 FROM EmployeeCount)
BEGIN
    INSERT INTO EmployeeCount (emp_count) VALUES (0);
END
```